



# Syscalls

Even though we are talking directly to the CPU through machine instructions in Assembly, we do not have to invoke every type of command using basic machine instructions only. Programs regularly use many kinds of operations. The Operating System can help us through syscalls to not have to execute these operations every time manually.

For example, suppose we need to write something on the screen, without syscalls. In that case, we will need to talk to the Video Memory and Video I/O, resolve any encoding required, send our input to be printed, and wait for the confirmation that it has been printed. As expected, if we had to do all of this to print a single character, it would make assembly codes much longer.

## Linux Syscall

A `syscall` is like a globally available function written in `C`, provided by the Operating System Kernel. A syscall takes the required arguments in the registers and executes the function with the provided arguments. For example, if we wanted to write something to the screen, we can use the `write` syscall, provide the string to be printed and other required arguments, and then call the syscall to issue the print.

There are many available syscalls provided by the Linux Kernel, and we can find a list of them and the `syscall number` of each one by reading the `unistd_64.h` system file:

```
shellsession
kimvutrinh@htb[/htb]$ cat /usr/include/x86_64-linux-gnu/asm/unistd_64.h
#ifndef _ASM_X86_UNISTD_64_H
#define _ASM_X86_UNISTD_64_H 1

#define __NR_read 0
#define __NR_write 1
#define __NR_open 2
#define __NR_close 3
#define __NR_stat 4
#define __NR_fstat 5
```

The above file sets the syscall number for each syscall to refer to that syscall using this number.

Note: With `32-bit` x86 processors, the syscall numbers are in the `unistd_32.h` file.

Let's practice using syscalls with the `write` syscall that prints to the screen. We will not be printing the Fibonacci numbers just yet, but will instead start by printing an intro message, `Fibonacci Sequence`, at the beginning of our program.

---

## Syscall Function Arguments

To use the `write` syscall, we must first know what arguments it accepts. To find the arguments accepted by a syscall, we can use the `man -s 2` command with the syscall name from the above list:

```
kimvutrinh@htb[/htb]$ man -s 2 write
...SNIP...
    ssize_t write(int fd, const void *buf, size_t count);
```

As we can see from the above output, the `write` function has the following syntax:

```
ssize_t write(int fd, const void *buf, size_t count);
```

We see that the syscall function expects `3` arguments:

1. File Descriptor `fd` to be printed to (*usually `1` for `stdout`* )
2. The address pointer to the string to be printed
3. The length we want to print

Once we provide these arguments, we can use the syscall instruction to execute the function and print to screen. In addition to these manual methods of locating syscalls and function arguments, there are online resources we can use to quickly look for syscalls, their numbers, and the arguments they expect, like [this table](#). Furthermore, we can always refer to the `Linux` source code on [Github](#).

Tip: The `-s 2` flag specifies `syscall` man pages. We can check `man man` to see various sections for each man page.

## Syscall Calling Convention

Now that we understand how to locate various syscall and their arguments let's start learning how to call them. To call a syscall, we have to:

1. Save registers to stack
2. Set its syscall number in `rax`
3. Set its arguments in the registers
4. Use the syscall assembly instruction to call it

We usually should save any registers we use to the stack before any function call or syscall. However, as we are running this syscall at the beginning of our program before using any registers, we don't have any values in the registers, so we should not worry about saving them.

We will discuss saving registers to the stack when we get to `Function Calls`.

## Syscall Number

Let's start by moving the syscall number to the `rax` register. As we saw earlier, the `write` syscall has a number `1`, so we can start with the following command:

```
mov rax, 1
```

nasm

Now, if we reach the syscall instruction, the Kernel would know which syscall we are calling.

## Syscall Arguments

Next, we should put each of the function's arguments in its corresponding register. The `x86_64` architecture's calling convention specifies in which register each argument should be placed (e.g., first arg should be in `rdi`). All functions and syscalls should follow this standard and take their

arguments from the corresponding registers. We have discussed the following table in the `Registers` section:

| Description                 | 64-bit Register  | 8-bit Register   |
|-----------------------------|------------------|------------------|
| Syscall Number/Return value | <code>rax</code> | <code>al</code>  |
| Callee Saved                | <code>rbx</code> | <code>bl</code>  |
| 1st arg                     | <code>rdi</code> | <code>dil</code> |
| 2nd arg                     | <code>rsi</code> | <code>sil</code> |
| 3rd arg                     | <code>rdx</code> | <code>d1</code>  |
| 4th arg                     | <code>rcx</code> | <code>c1</code>  |
| 5th arg                     | <code>r8</code>  | <code>r8b</code> |
| 6th arg                     | <code>r9</code>  | <code>r9b</code> |

As we can see, we have a register for each of the first `6` arguments. Any additional arguments can be stored in the stack (though not many syscalls use more than `6` arguments.).

Note: `rax` is also used for storing the `return value` of a syscall or a function. So, if we were expecting to get a value back from a syscall/function, it will be in `rax` .

With that, we should know our arguments and in which register we should store them. Going back to the `write` syscall function, we should pass: `fd` , `pointer` , and `length` . We can do so as follows:

1. `rdi` -> `1` (for stdout)
2. `rsi` -> `'Fibonacci Sequence:\n'` (pointer to our string)
3. `rdx` -> `20` (length of our string)

We can use `mov rcx, 'string'` . However, we can only store up to 8 bytes in a register (i.e., 64 bits), which is equivalent to 8 ASCII characters, assuming each character is 1 byte, so our intro string would not fit. Instead, let's create a variable with our string (as we learned in the [Assembly File Structure](#) section), similarly to what we did with the [Hello World](#) program:

```
global _start

section .data
    message db "Fibonacci Sequence:", 0x0a
```

Note how we added `0x0a` after our string, to add a new line character.

The `message` label is a pointer to where our string will be stored in the memory. So, we can use it as our second argument. So, our final syscall code should be as follows:

```
mov rax, 1      ; rax: syscall number 1
mov rdi, 1      ; rdi: fd 1 for stdout
mov rsi,message ; rsi: pointer to message
mov rdx, 20     ; rdx: print length of 20 bytes
```

Tip: If we ever needed to create a pointer to a value stored in a register, we can simply push it to the stack, and then use the `rsp` pointer to point to it.

We may also use a dynamically calculated `length` variable by using `equ`, similarly to what we did with the `Hello World` program.

## Calling Syscall

Now that we have our syscall number and arguments in place, the only thing left is to do the syscall instruction. So, let's add a syscall instruction and add the instructions to the beginning of our `fib.s` code, which should look as follows:

```
global _start

section .data
    message db "Fibonacci Sequence:", 0x0a

section .text
_start:
    mov rax, 1      ; rax: syscall number 1
    mov rdi, 1      ; rdi: fd 1 for stdout
    mov rsi, message ; rsi: pointer to message
    mov rdx, 20     ; rdx: print length of 20 bytes
    syscall         ; call write syscall to the intro message
    xor rax, rax    ; initialize rax to 0
    xor rbx, rbx    ; initialize rbx to 0
    inc rbx         ; increment rbx to 1
loopFib:
```

nasm

```

add rax, rbx      ; get the next number
xchg rax, rbx     ; swap values
cmp rbx, 10       ; do rbx - 10
js loopFib        ; jump if result is <0

```

Let's now assemble our code and run it, and see if our intro message gets printed:

```

kimvutrinh@htb[/htb]$ ./assembler.sh fib.s

```

```

Fibonacci Sequence:
[1] 107348 segmentation fault ./fib

```

We see that indeed our string is printed to the screen. Let's run it through `gdb`, and break at the syscall to see how all arguments are setup before we call syscall, as follows:

```

$ gdb -q ./fib
gef> disas _start
Dump of assembler code for function _start:
..SNIP...
0x0000000000401011 <+17>:  syscall
gef> b *_start+17
Breakpoint 1 at 0x401011
gef> r

registers ——— $rax : 0x1
$rbx : 0x0
$rcx : 0x0
$rdx : 0x14
$rsp : 0x00007fffffffe410 → 0x0000000000000001
$rbp : 0x0
$rsi : 0x0000000000402000 → "Fibonacci Sequence:\n"$rdi : 0x1

```

```
gef> si
```

```
Fibonacci Sequence:
```

We see a couple of things that we expected:

1. Our arguments are properly set in the corresponding registers before each syscall.
2. A pointer to our message is loaded in `rsi`.

Now, we have successfully used the `write` syscall to print our intro message.

## Exit Syscall

Finally, since we have understood how syscalls work, let's go through another essential syscall used in programs: `Exit syscall`. We may have noticed that so far, whenever our program finishes executing, it exits with a `segmentation fault`, as we just saw when we ran `./fib`. This is because we are ending our program abruptly, without going through the proper procedure of exiting programs in Linux, by calling the `exit syscall` and passing an exit code.

So, let's add this to the end of our code. First, we need to find the `exit syscall` number, as follows:

```
shellsession
kimvutrinh@htb[/htb]$ grep exit /usr/include/x86_64-linux-gnu/asm/unistd_64.h
```

```
#define __NR_exit 60
#define __NR_exit_group 231
```

We need to use the first one, with a syscall number `60`. Next, let's see if the `exit syscall` needs any arguments:

```
kimvutrinh@htb[/htb]$ man -s 2 exit
```

...SNIP...

```
void _exit(int status);
```

We see that it only needs one integer argument, `status`, which is explained to be the exit code. In Linux, whenever a program exits without any errors, it passes an exit code of `0`. Otherwise, the exit code is a different number, usually `1`. In our case, as everything went as expected, we'll pass the exit code of `0`. Our `exit syscall` code should be as follows:

```
mov rax, 60
mov rdi, 0
syscall
```

Now, let's add it to the end of our code:

```
global _start

section .data
    message db "Fibonacci Sequence:", 0x0a

section .text
_start:
```

```

    mov rax, 1      ; rax: syscall number 1
    mov rdi, 1      ; rdi: fd 1 for stdout
    mov rsi,message ; rsi: pointer to message
    mov rdx, 20     ; rdx: print length of 20 bytes
    syscall         ; call write syscall to the intro message
    xor rax, rax    ; initialize rax to 0
    xor rbx, rbx    ; initialize rbx to 0
    inc rbx         ; increment rbx to 1
loopFib:
    add rax, rbx    ; get the next number
    xchg rax, rbx   ; swap values
    cmp rbx, 10    ; do rbx - 10
    js loopFib     ; jump if result is <0
    mov rax, 60
    mov rdi, 0
    syscall

```

We can now assemble our code and rerun it:

```

kimvutrinh@htb[/htb]$ ./assembler.sh fib.s

```

Fibonacci Sequence:

Great! We see that this time our program exited properly without a **segmentation fault**. We can check the exit code that was passed as follows:

```

kimvutrinh@htb[/htb]$ echo $?

```

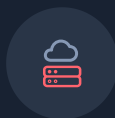
0

As expected, the exit code was 0, as we specified in our syscall.

Practice: To get a full grasp of how syscalls work, try to implement the write syscall to print the Fibonacci number, and place it after 'xchg rax, rbx'.

Spoiler: It will not work. Try to find out why, and attempt to fix it to print the first few Fibonacci numbers below 10 (hint: use ASCII).

## Connect to HTB



### Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline



Enable step-by-step solutions ⓘ

PRO

### Optional Exercise 1

xp +40 ^

What is the syscall number of "execve"?

Write your answer

Submit

💡 Reveal Answer

Optional Exercise 2

XP +40 ^

How many arguments does "execve" take?

Write your answer

Submit

💡 Reveal Answer

← Previous

Section 17 / 24

XP +20

Next

Mark Complete & Next